DATE: EXERCISE NO: EXERCISE NAME:

WEEK-1: HTML Frames: Frame Structure: Design a webpage for the online book store using HTML frames. Include a navigation frame, a content frame for displaying book details, and a footer frame for additional information.

Code:

WEEK1.html:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Online Book Store</title>
  </head>
  <frameset rows="20%,70%,10%" border="2">
    <iframe src=""></iframe>
    <frame src="navigation.html" name="navFrame">
    <frame src="content.html" name="contentFrame">
    <frame src="footer.html" name="footerFrame">
  </frameset>
</html>
```

Navigation.html:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Navigation</title>
  </head>
  <body bgcolor="BBBBBB">
    <h3>Book Categories</h3>
    <h1>Hello World Welcome to VNRVJIET</h1>
    <ul>
      <li><a href="https://www.microsoft.com" target="contentFrame">Computer Science</a></li>
      <li><a href="https://www.ieee.org target="contentFrame">Electronics</a></li>
      <li><a href="content.html" target="contentFrame">Mechanical</a></li>
      <li><a href="content.html" target="contentFrame">Civil</a></li>
    </ul>
  </body>
</html>
```

Footer.html:

```
<!DOCTYPE html>
<head>
  <title>Footer</title>
</head>
<body bgcolor="90EE90" align="center">
  <p>2025 Online Book Store | Contact: Support@bookstore.com</p>
</body>
</html>
```

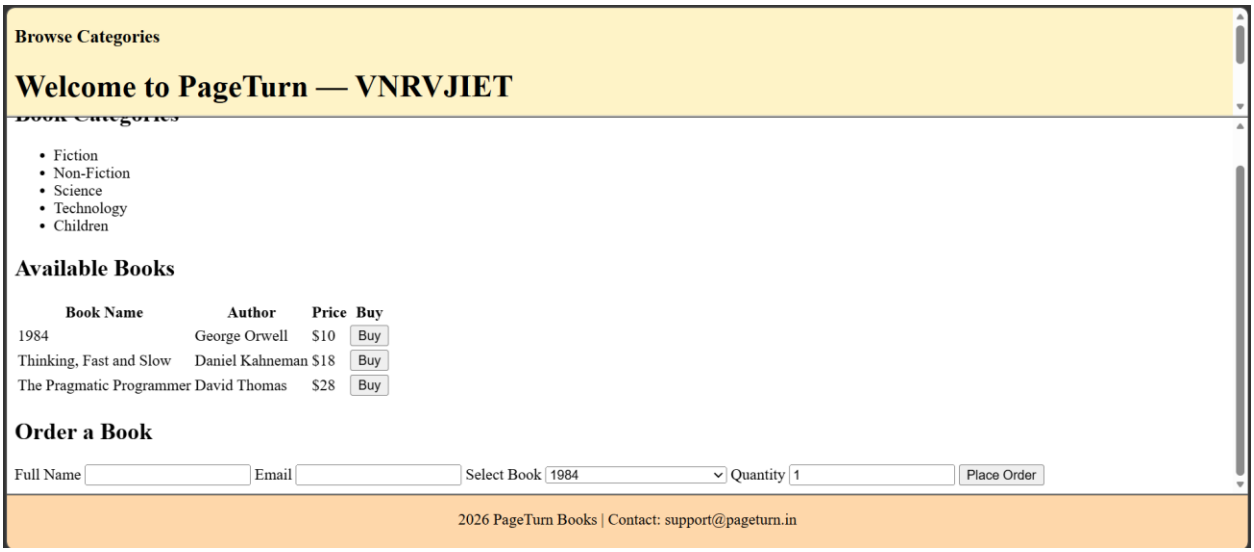
Content.html:

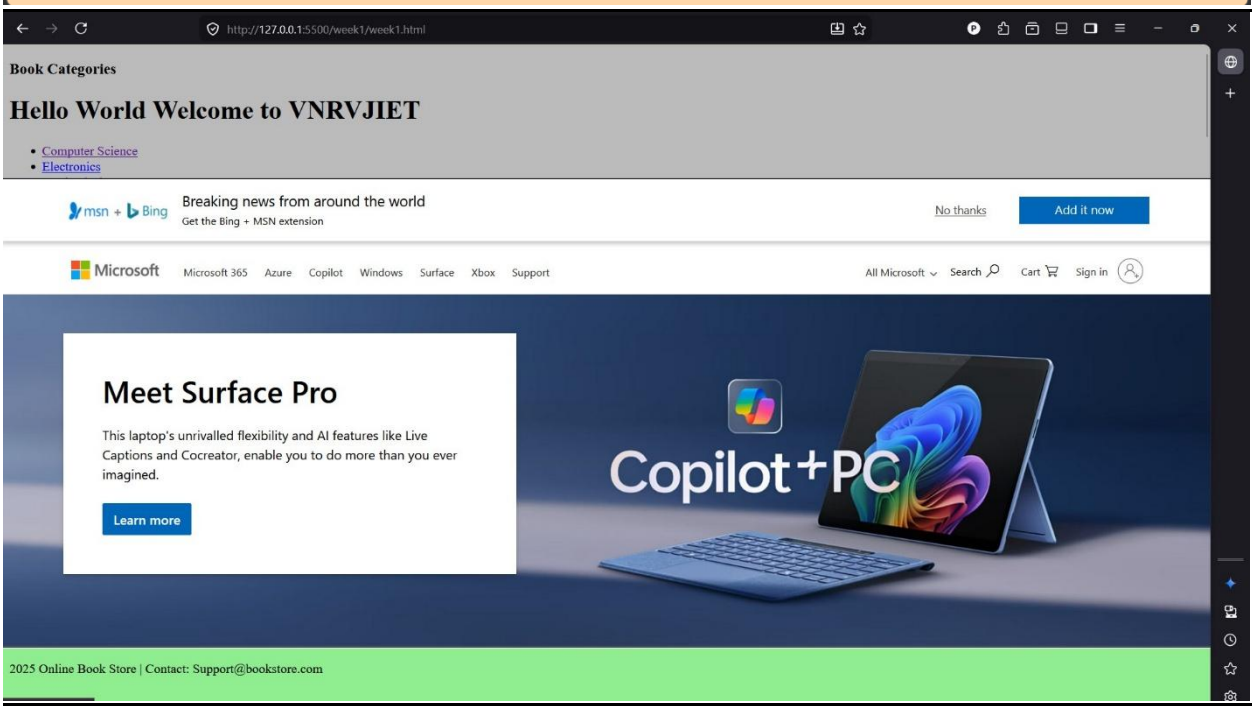
```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Book Store</title>
   <link rel="stylesheet" href="content.css">
</head>
<body>

   <section class="categories">
      <h2>Book Categories</h2>
      <ul>
         <li>Fiction</li>
         <li>Non-Fiction</li>
         <li>Science</li>
         <li>Technology</li>
         <li>Children</li>
      </ul>
   </section>

   <section class="books">
      <h2>Available Books</h2>
      <table>
         <tr>
            <th>Book Name</th>
            <th>Author</th>
            <th>Price</th>
            <th>Buy</th>
         </tr>
         <tr>
            <td>The Alchemist</td>
            <td>Paulo Coelho</td>
            <td>$12</td>
            <td><button>Buy</button></td>
         </tr>
         <tr>
            <td>Atomic Habits</td>
            <td>James Clear</td>
            <td>$15</td>
            <td><button>Buy</button></td>
         </tr>
         <tr>
            <td>Clean Code</td>
            <td>Robert C. Martin</td>
            <td>$20</td>
```

```
            <td><button>Buy</button></td>
          </tr>
        </table>
    </section>

    <section class="order">
        <h2>Order a Book</h2>
        <form>
            <label>Full Name</label>
            <input type="text" required>

            <label>Email</label>
            <input type="email" required>

            <label>Select Book</label>
            <select>
                <option>The Alchemist</option>
                <option>Atomic Habits</option>
                <option>Clean Code</option>
            </select>

            <label>Quantity</label>
            <input type="number" min="1" value="1">

            <button type="submit">Place Order</button>
        </form>
    </section>

</body>
</html>
```

WEBPAGE:

DATE: EXERCISE NO: EXERCISE NAME:





DATE: EXERCISE NO: EXERCISE NAME:

WEEK-2: CSS Styling: Apply CSS styling to enhance the visual appeal of the frames and content. Use appropriate styles for fonts, colors, and layout.

Week2.html:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Online Book Store</title>
   </head>
   <frameset rows="20%,70%,10%" border="2">
      <iframe src=""></iframe>
      <frame src="navigation.html" name="navFrame">
      <frame src="content.html" name="contentFrame">
      <frame src="footer.html" name="footerFrame">
   </frameset>
</html>
```

Navigation.html:

```
<!DOCTYPE html>
<html>
   <head>
      <title>Navigation</title>
   </head>
   <body bgcolor="BBBBBB">
      <h3>Book Categories</h3>
      <h1>Hello World Welcome to VNRVJIET</h1>
      <ul>
         <li><a href="https://www.google.com" target="contentFrame">Computer Science</a></li>
         <li><a href="content.html" target="contentFrame">Electronics</a></li>
         <li><a href="content.html" target="contentFrame">Mechanical</a></li>
         <li><a href="content.html" target="contentFrame">Civil</a></li>
      </ul>
   </body>
</html>
```

Navigation.css:

```
body {
   font-family: Arial, sans-serif;
   background-color: #eeeeee;
   padding: 10px;
}

h3 {
   color: #2c3e50;
}

ul {
   padding-left: 15px;
```

```
}

li {
   margin: 6px 0;
}

a {
   text-decoration: none;
   color: #2980b9;
}

a:hover {
   text-decoration: underline;
}
```

Footer.html:

```
<!DOCTYPE html>
<head>
   <title>Footer</title>
</head>
<body bgcolor="90EE90" align="center">
   <p>2025 Online Book Store | Contact: Support@bookstore.com</p>
</body>
</html>
```

Footer.css:

```
body {
   font-family: Arial, sans-serif;
   background-color: #90EE90;
   margin: 0;
   text-align: center;
}

p {
   font-size: 14px;
   padding-top: 10px;
}
```

Content.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Book Store</title>
   <link rel="stylesheet" href="content.css">
</head>
<body>

   <!-- Categories List -->
```

```
<section class="categories">
  <h2>Book Categories</h2>
  <ul>
    <li>Fiction</li>
    <li>Non-Fiction</li>
    <li>Science</li>
    <li>Technology</li>
    <li>Children</li>
  </ul>
</section>

<!-- Books Table -->
<section class="books">
  <h2>Available Books</h2>
  <table>
    <tr>
      <th>Book Name</th>
      <th>Author</th>
      <th>Price</th>
      <th>Buy</th>
    </tr>
    <tr>
      <td>The Alchemist</td>
      <td>Paulo Coelho</td>
      <td>$12</td>
      <td><button>Buy</button></td>
    </tr>
    <tr>
      <td>Atomic Habits</td>
      <td>James Clear</td>
      <td>$15</td>
      <td><button>Buy</button></td>
    </tr>
    <tr>
      <td>Clean Code</td>
      <td>Robert C. Martin</td>
      <td>$20</td>
      <td><button>Buy</button></td>
    </tr>
  </table>
</section>

<!-- Order Form -->
<section class="order">
  <h2>Order a Book</h2>
  <form>
    <label>Full Name</label>
    <input type="text" required>
```

```
        <label>Email</label>
        <input type="email" required>

        <label>Select Book</label>
        <select>
          <option>The Alchemist</option>
          <option>Atomic Habits</option>
          <option>Clean Code</option>
        </select>

        <label>Quantity</label>
        <input type="number" min="1" value="1">

        <button type="submit">Place Order</button>
      </form>
    </section>

</body>
</html>
```

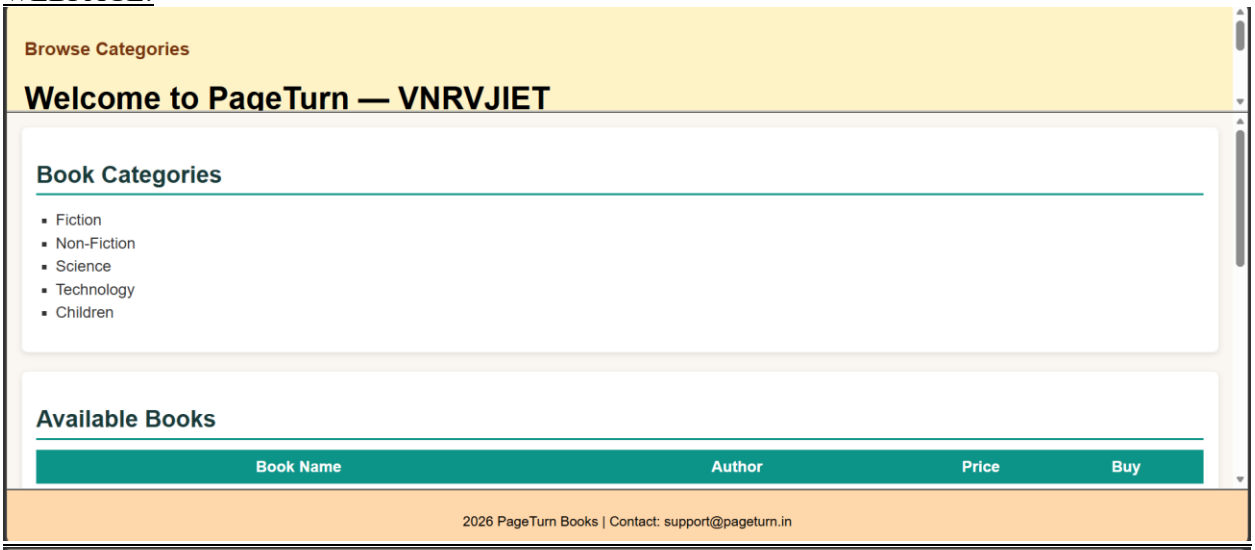
Content.css:

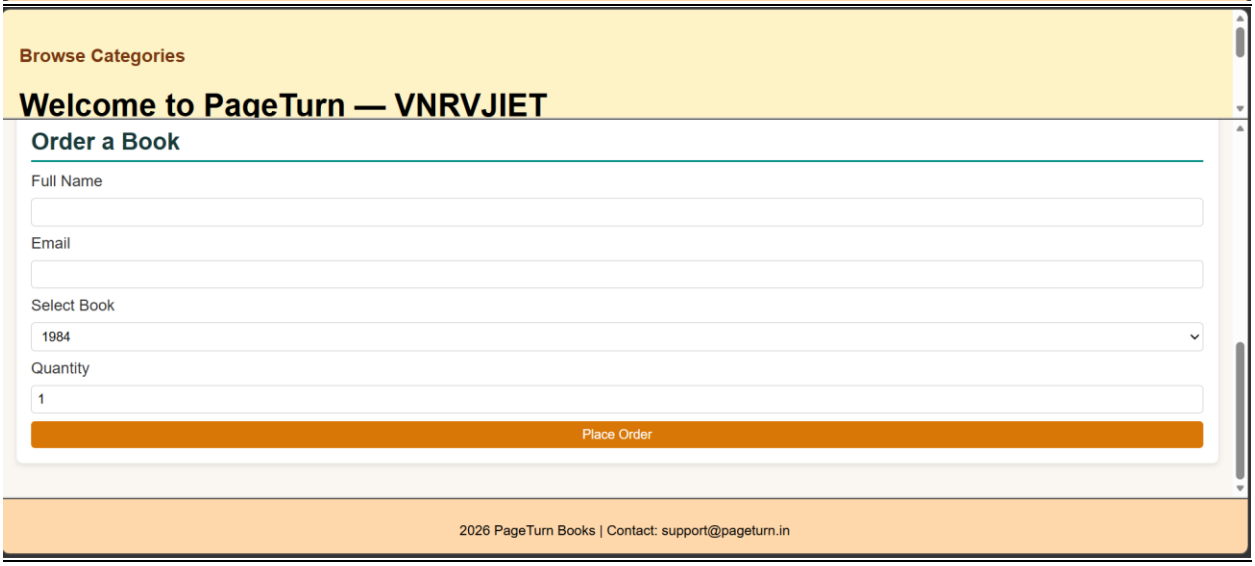
```
/* General page styling */
body {
  font-family: Arial, sans-serif;
  background-color: #f5f7fa;
  margin: 0;
  padding: 15px;
  color: #333;
}

/* Section styling */
section {
  background: #ffffff;
  padding: 15px;
  margin-bottom: 20px;
  border-radius: 6px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
}

/* Headings */
h2 {
  color: #2c3e50;
  margin-bottom: 10px;
  border-bottom: 2px solid #3498db;
  padding-bottom: 5px;
}

/* Categories list */
.categories ul {
```

```
    list-style-type: square;
    padding-left: 20px;
}

.categories li {
    margin: 5px 0;
}
/* Table styling */
table {
    width: 100%;
    border-collapse: collapse;
}

th {
    background-color: #3498db;
    color: white;
    padding: 8px;
}

td {
    padding: 8px;
    border-bottom: 1px solid #ddd;
}

/* Buttons */
button {
    background-color: #2ecc71;
    color: white;
    border: none;
    padding: 6px 12px;
    border-radius: 4px;
    cursor: pointer;
}

button:hover {
    background-color: #27ae60;
}

/* Form styling */
form {
    display: grid;
    row-gap: 8px;
}

input, select {
    padding: 6px;
    border: 1px solid #ccc;
    border-radius: 4px;
}
```

DATE: EXERCISE NO: EXERCISE NAME:

WEBPAGE:





WEEK-3:
JavaScript:
a) User Registration Form
Create a registration form for users signing up on the online bookstore. Validate the following:
• Username: Should not be empty and must be at least 5 characters long.
• Password: Should not be empty and must be at least 8 characters long.
• Confirm Password: Should match the entered password.
b) Forgot Password Form
Implement a form for users to reset their password. Validate the following:
• Email Address: Should be a valid email format.
• Provide feedback if the email address is not associated with any account.
c) Dynamically Content Loading
Use JavaScript to dynamically load book details into the content frame when a user clicks on a book in the navigation frame.

Week3.html:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Online Book Store - Week 3</title>
  </head>
  <frameset rows="20%,70%,10%" border="2">
    <frame src="navigation.html" name="navFrame">
    <frame src="content.html" name="contentFrame">
    <frame src="footer.html" name="footerFrame">
  </frameset>
</html>
```

Week3.js:

```
// week3.js - Dynamically load book details into the content frame

document.addEventListener('DOMContentLoaded', function () {
  // This script runs inside the navigation frame (navigation.html)
  var navLinks = document.querySelectorAll('a[data-book-id]');
  if (!navLinks.length) {
    return;
  }

  var bookData = {
    'clean-code': {
      title: 'Clean Code',
      author: 'Robert C. Martin',
      category: 'Technology / Programming',
      price: '$20',
      isbn: '978-0132350884',
```

```
        availability: 'In Stock',
        description: 'A handbook of agile software craftsmanship focusing on writing clean, maintainable code.'
    },
    'alchemist': {
        title: 'The Alchemist',
        author: 'Paulo Coelho',
        category: 'Fiction',
        price: '$12',
        isbn: '978-0061122415',
        availability: 'In Stock',
        description: 'A novel about a shepherd boy on a journey to realize his personal legend.'
    },
    'atomic-habits': {
        title: 'Atomic Habits',
        author: 'James Clear',
        category: 'Self Help',
        price: '$15',
        isbn: '978-0735211292',
        availability: 'Limited Stock',
        description: 'Practical strategies for building good habits and breaking bad ones.'
    },
    'ds-algo': {
        title: 'Data Structures and Algorithms',
        author: 'Narasimha Karumanchi',
        category: 'Computer Science',
        price: '$25',
        isbn: '978-8193245279',
        availability: 'Out of Stock',
        description: 'Detailed explanations of core data structures and algorithms with interview focus.'
    }
};

function loadBookIntoContentFrame(bookId) {
    var info = bookData[bookId];
    if (!info) {
        return;
    }

    var contentFrame = parent.frames['contentFrame'];
    if (!contentFrame || !contentFrame.document) {
        return;
    }
```

```
        var doc = contentFrame.document;
        var container = doc.getElementById('bookDetails');
        if (!container) {
            return;
        }

        container.innerHTML =
            '<h3>' + info.title + '</h3>' +
            '<p><strong>Author:</strong> ' + info.author + '</p>' +
            '<p><strong>Category:</strong> ' + info.category + '</p>' +
            '<p><strong>ISBN:</strong> ' + info.isbn + '</p>' +
            '<p><strong>Price:</strong> ' + info.price + '</p>' +
            '<p><strong>Availability:</strong> ' + info.availability + '</p>' +
            '<p><strong>Description:</strong> ' + info.description + '</p>';
    }

    navLinks.forEach(function (link) {
        link.addEventListener('click', function (e) {
            e.preventDefault();
            var bookId = this.getAttribute('data-book-id');
            loadBookIntoContentFrame(bookId);
        });
    });
});
```

Registration.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Registration - Online Bookstore</title>
    <link rel="stylesheet" href="registration.css">
</head>
<body>
    <div class="container">
        <h2>User Registration</h2>
        <form id="registrationForm" autocomplete="off">
            <label for="username">Username:</label>
            <input type="text" id="username" name="username" required>

            <label for="password">Password:</label>
            <input type="password" id="password" name="password" required>

            <label for="confirmPassword">Confirm Password:</label>
            <input type="password" id="confirmPassword" name="confirmPassword" required>
```

```
            <div id="errorMsg" class="error" style="display:none;"></div>
            <button type="submit">Register</button>
            <div style="text-align:center; margin-top:15px;">
                <a href="forgotpassword.html">Forgot Password?</a>
            </div>
        </form>
    </div>
    <script src="registration.js"></script>
</body>
</html>
```

ForgotPassword.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Forgot Password - Online Bookstore</title>
    <link rel="stylesheet" href="forgotpassword.css">
</head>
<body>
    <div class="container">
        <h2>Forgot Password</h2>
        <form id="forgotPasswordForm" autocomplete="off">
            <label for="email">Email Address:</label>
            <input type="email" id="email" name="email" required>
            <div id="fpErrorMsg" class="error" style="display:none;"></div>
            <button type="submit">Reset Password</button>
        </form>
    </div>
    <script src="forgotpassword.js"></script>
</body>
</html>
```

Forgotpassword.js:

```
document.addEventListener('DOMContentLoaded', function() {
    var form = document.getElementById('forgotPasswordForm');
    if (!form) return;
    form.addEventListener('submit', function(e) {
        e.preventDefault();
        var email = document.getElementById('email').value.trim();
        var errorMsg = document.getElementById('fpErrorMsg');
        errorMsg.style.display = 'none';
        errorMsg.innerText = '';
```

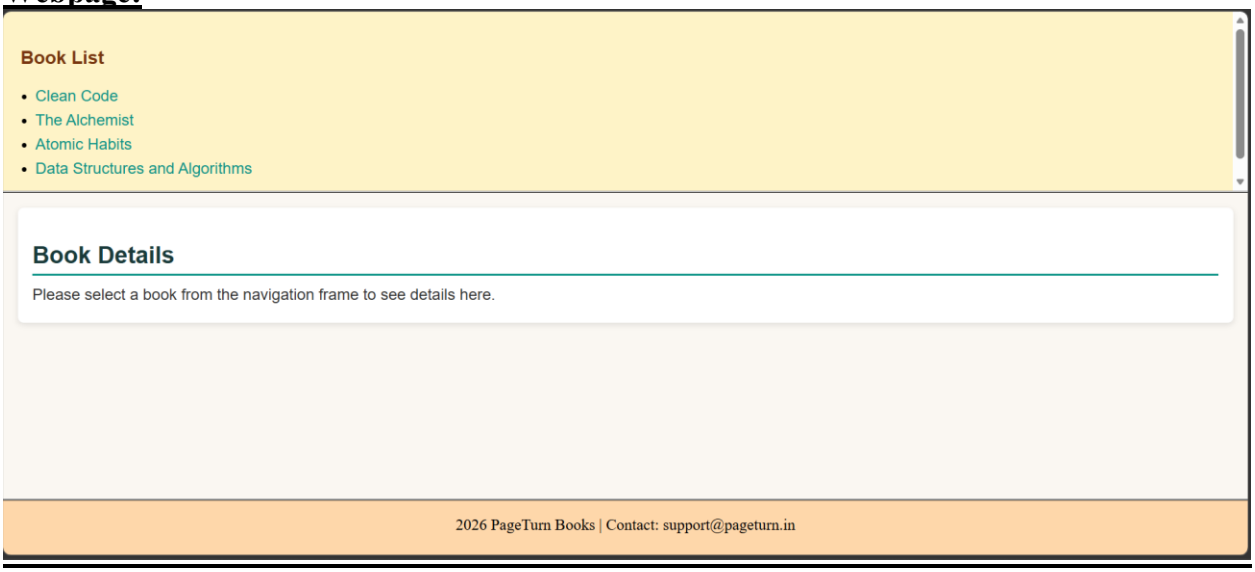
```
        var emailRegex = /^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$/;
        if (!emailRegex.test(email)) {
            errorMsg.innerText = 'Please enter a valid email address.';
            errorMsg.style.display = 'block';
            return;
        }

        // Simulated check for email association (for demo, only these emails are valid)
        var validEmails = ['user1@example.com', 'user2@example.com',
'test@bookstore.com','pranav.tungani@gmail.com'];
        if (!validEmails.includes(email)) {
            errorMsg.innerText = 'This email address is not associated with any account.';
            errorMsg.style.display = 'block';
            return;
        }

        alert('A password reset link has been sent to your email.');
        form.reset();
    });
});
```

Registration.js:

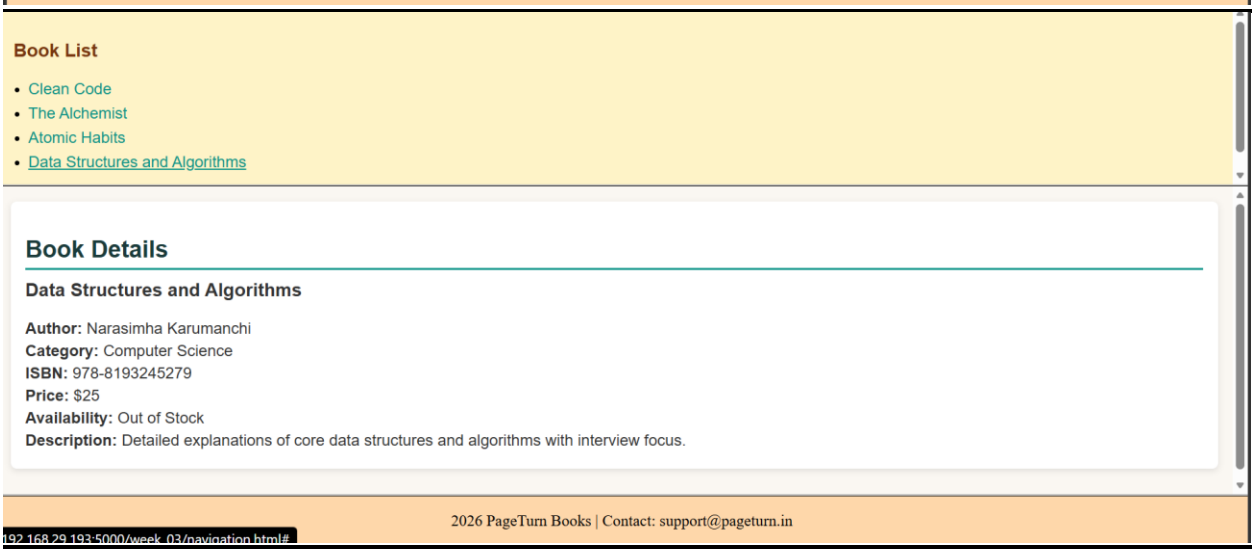
```
// registration.js - Security and validation for registration form

document.addEventListener('DOMContentLoaded', function() {
    var form = document.getElementById('registrationForm');
    if (!form) return;
    form.addEventListener('submit', function(e) {
        e.preventDefault();
        var username = document.getElementById('username').value.trim();
        var password = document.getElementById('password').value;
        var confirmPassword = document.getElementById('confirmPassword').value;
        var errorMsg = document.getElementById('errorMsg');
        errorMsg.style.display = 'none';
        errorMsg.innerText = '';

        // Username validation
        if (username.length < 5) {
            errorMsg.innerText = 'Username must be at least 5 characters long.';
            errorMsg.style.display = 'block';
            return;
        }
        // Password validation
        if (password.length < 8) {
            errorMsg.innerText = 'Password must be at least 8 characters long.';
            errorMsg.style.display = 'block';
```

```
        return;
    }
    // Password strength (at least 1 uppercase, 1 lowercase, 1 number)
    var strongRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).+$/;
    if (!strongRegex.test(password)) {
        errorMsg.innerText = 'Password must contain uppercase, lowercase, and a number.';
        errorMsg.style.display = 'block';
        return;
    }
    // Confirm password
    if (password !== confirmPassword) {
        errorMsg.innerText = 'Passwords do not match.';
        errorMsg.style.display = 'block';
        return;
    }
    // All validations passed
    alert('Registration successful!');
    form.reset();

    // Navigate to the content page after successful registration
    window.location.href = 'navigation.html';
  });
});
```

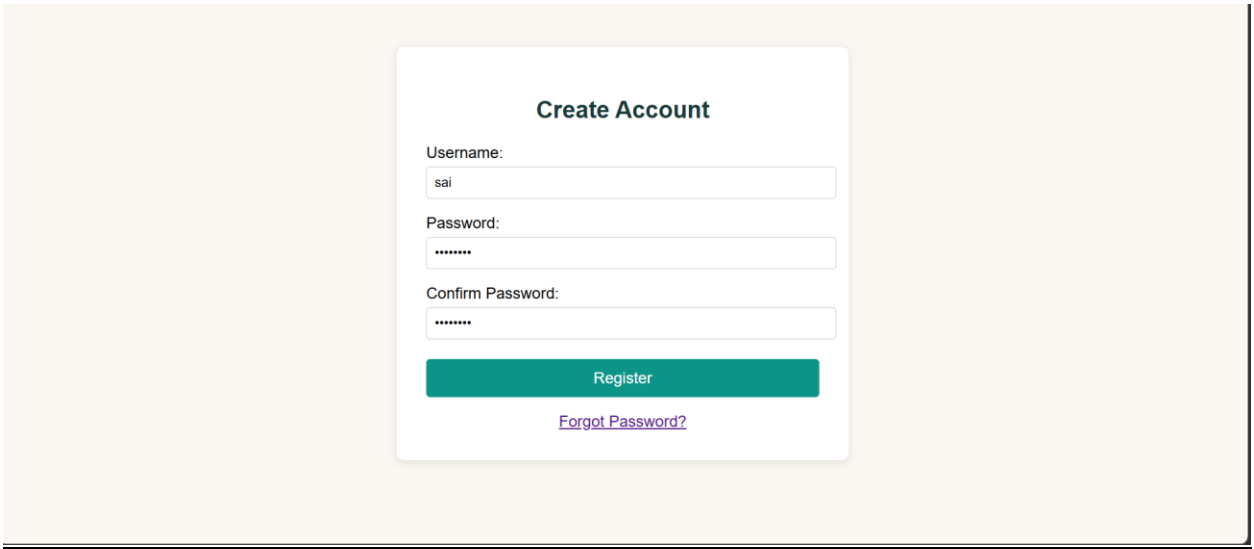
Webpage:

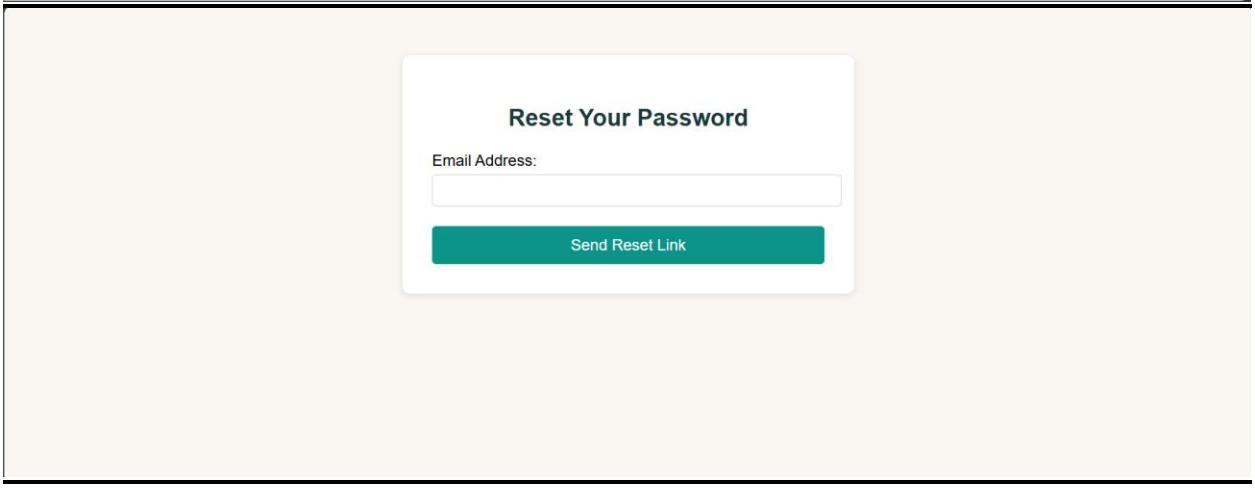


DATE: EXERCISE NO: EXERCISE NAME:



DATE: EXERCISE NO: EXERCISE NAME:





WEEK-4: Shopping Cart Functionality: Implement a simple shopping cart using JavaScript. Allow users to add books to their cart and view the cart's contents.

Cart.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>My Cart</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
   <main>
   <h1>My Cart</h1>
   <a href="index.html">Back to Store</a>

   <div id="cart-items"></div>
   <p><strong>Total: Rs. <span id="total">0</span></strong></p>
   <button onclick="clearCart()">Clear Cart</button>
   </main>

   <script src="script.js"></script>
</body>
</html>
```

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Book Store</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
   <main>
   <h1>Book Store</h1>
   <a href="cart.html">View Cart (<span id="cart-count">0</span>)</a>

   <div id="books">
      <div class="book">
      <h3>The Alchemist</h3>
      <p>By Paulo Coelho</p>
      <p>Price: Rs. 299</p>
      <button onclick="addToCart('The Alchemist', 299)">Add to Cart</button>
   </div>
```

```
    <div class="book">
        <h3>Atomic Habits</h3>

        <p>By James Clear</p>
        <p>Price: Rs. 499</p>
        <button onclick="addToCart('Atomic Habits', 499)">Add to Cart</button>
    </div>

    <div class="book">
        <h3>Rich Dad Poor Dad</h3>
        <p>By Robert Kiyosaki</p>
        <p>Price: Rs. 349</p>
        <button onclick="addToCart('Rich Dad Poor Dad', 349)">Add to Cart</button>
    </div>

    <div class="book">
        <h3>Deep Work</h3>
        <p>By Cal Newport</p>
        <p>Price: Rs. 399</p>
        <button onclick="addToCart('Deep Work', 399)">Add to Cart</button>
    </div>
    </div>
    </main>

    <script src="script.js"></script>
</body>
</html>
```

Script.js:

```
// Get cart from localStorage (or empty array)
function getCart() {
    return JSON.parse(localStorage.getItem('cart')) || [];
}

// Save cart to localStorage
function saveCart(cart) {
    localStorage.setItem('cart', JSON.stringify(cart));
}

// Add a book to the cart
function addToCart(title, price) {
    var cart = getCart();
    cart.push({ title: title, price: price });
    saveCart(cart);
    alert(title + ' added to cart!');
    updateCount();
```

```
}

// Update the cart count shown on the store page
function updateCount() {
    var countEl = document.getElementById('cart-count');
    if (countEl) {
        countEl.textContent = getCart().length;
    }
}

// Show cart items on cart.html
function showCart() {
    var cart = getCart();
    var container = document.getElementById('cart-items');
    if (!container) return;

    if (cart.length === 0) {
        container.innerHTML = '<p>Your cart is empty.</p>';
        document.getElementById('total').textContent = 0;
        return;
    }

    var html = '<ul>';
    var total = 0;
    for (var i = 0; i < cart.length; i++) {
        html += '<li>' + cart[i].title + ' - Rs. ' + cart[i].price + '</li>';
        total += cart[i].price;
    }
    html += '</ul>';
    container.innerHTML = html;
    document.getElementById('total').textContent = total;
}

// Clear all items from cart
function clearCart() {
    localStorage.removeItem('cart');
    showCart();
}

// Run on page load
updateCount();
showCart();
```

Styles.css:

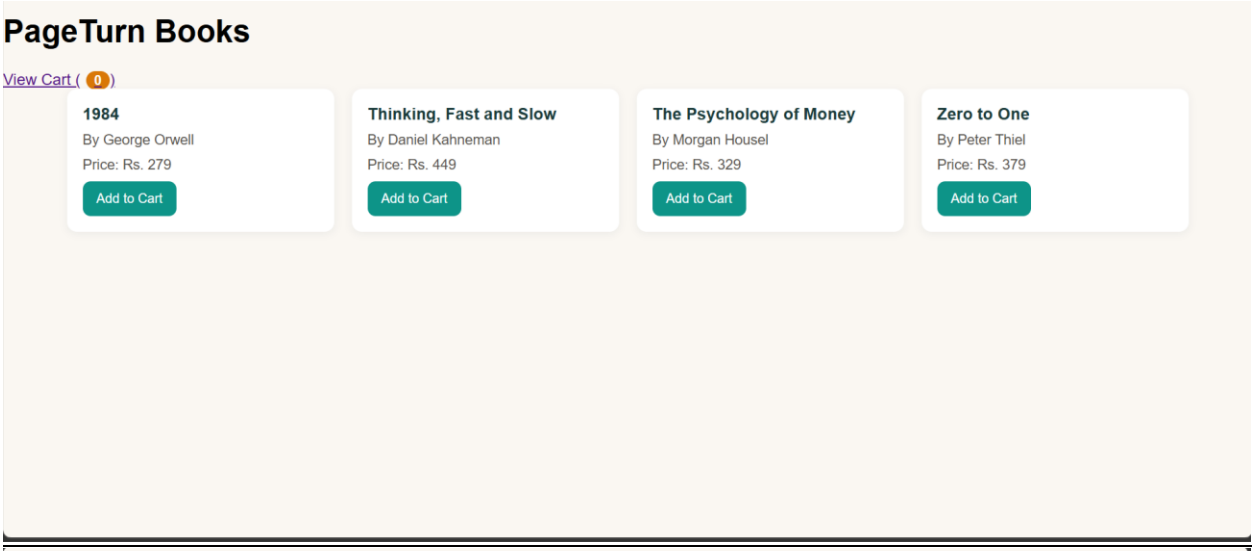
```
body {
    font-family: Arial, sans-serif;
```

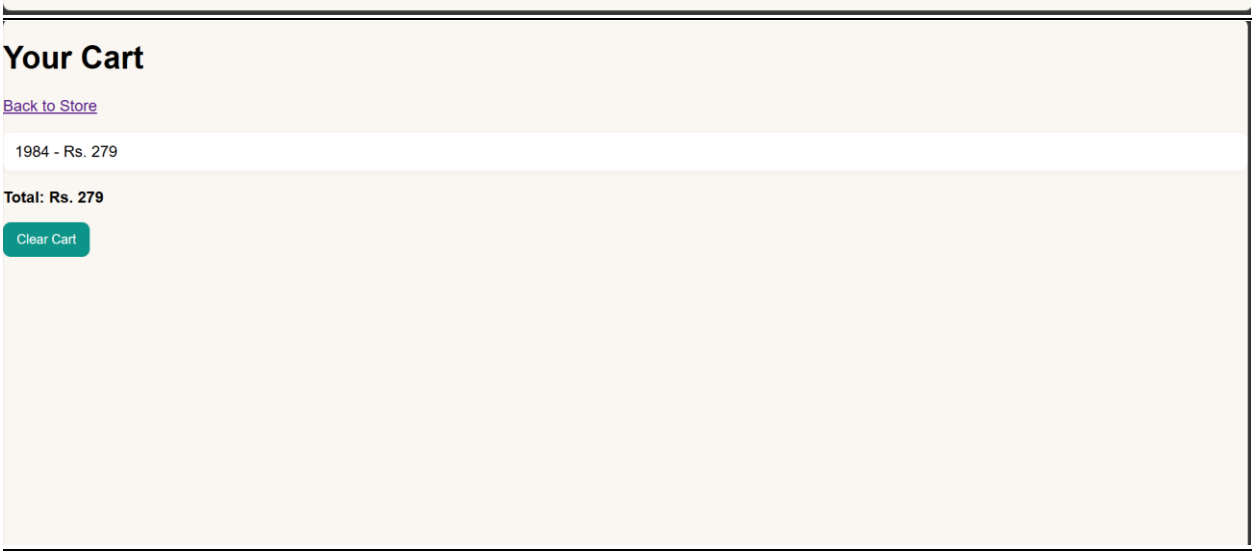
```
    margin: 0;
    padding: 0;
    background: #f5f7fb;
}
/* Books grid */
#books{
    display: flex;
    flex-wrap: wrap;
    gap: 18px;
    justify-content: center;
}

.book {
    background: #ffffff;
    border-radius: 10px;
    box-shadow: 0 2px 8px rgba(11,37,69,0.06);
    padding: 16px;
    width: 240px;
    display: flex;
    flex-direction: column;
    gap: 8px;
    transition: transform 160ms ease, box-shadow 160ms ease;
}

.book:hover {
    transform: translateY(-6px);
    box-shadow: 0 10px 22px rgba(11,37,69,0.10);
}

.book h3 { margin: 0; font-size: 1.05rem; color: #071b2d; }
.book p { margin: 0; color: #475569; font-size: 0.9rem; }
.book .price { font-weight: 700; color: #111827; }

button {
    padding: 10px 14px;
    background-color: #0b74de;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    align-self: start;
    transition: background-color 120ms ease, transform 120ms ease;
}

button:hover { background-color: #075fae; transform: translateY(-2px); }
```

```
button:active { transform: translateY(0); }

/* Cart link badge */
#cart-count {
   background: #ef4444;
   color: #fff;
   padding: 2px 8px;
   border-radius: 999px;
   margin-left: 6px;
   font-weight: 700;
   font-size: 0.85rem;
}

/* Cart page styles */
#cart-items { margin: 18px 0; }
.cart-item {
   display: flex;
   justify-content: space-between;
   align-items: center;
   padding: 10px 12px;
   background: #fff;
   border-radius: 8px;
   box-shadow: 0 1px 4px rgba(11,37,69,0.04);
   margin-bottom: 10px;
}

.cart-item .name { font-weight: 600; }
.cart-item .price { color: #0b2545; font-weight: 700; }

/* Responsive */
@media (max-width: 600px) {
   main { margin: 18px; padding: 12px; }
   .book { width: 100%; max-width: 420px; }
}

/* Small utility */
.muted { color: #6b7280; font-size: 0.9rem; }

/* Style cart list rendered by script.js */
#cart-items ul { list-style: none; padding: 0; margin: 0; }
#cart-items ul li {
   padding: 10px 12px;
   background: #fff;
   border-radius: 8px;
   box-shadow: 0 1px 4px rgba(11,37,69,0.04);
```

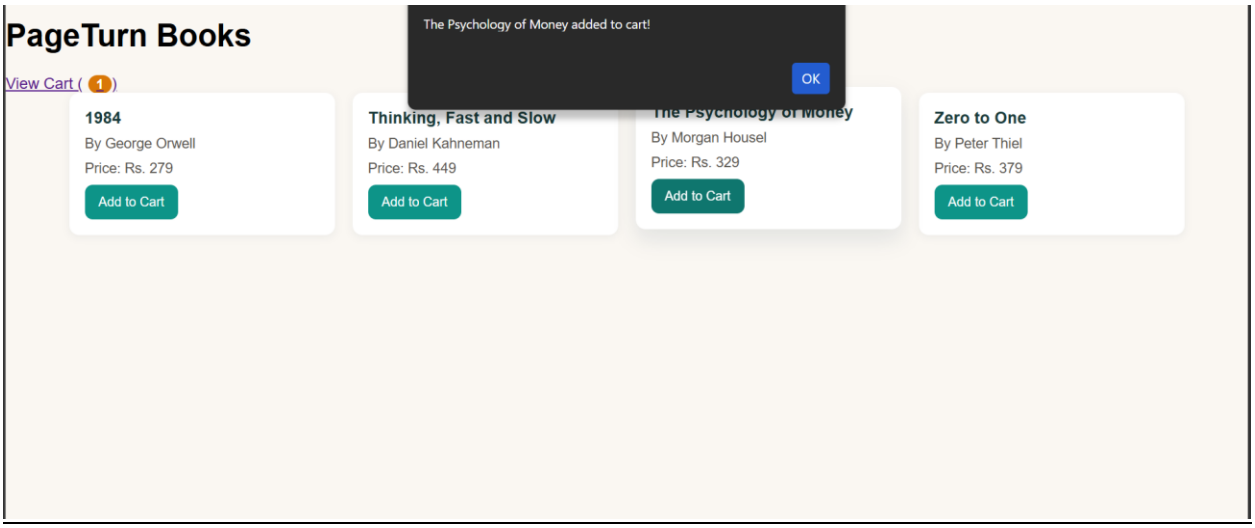
```
    margin-bottom: 10px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
#cart-items ul li::after { content: attr(data-price); font-weight:700; color:#0b2545; }
```

WEBPAGE:





DATE: EXERCISE NO: EXERCISE NAME:



WEEK-5: XML Structure and Design: Book Catalog XML: Design an XML structure to represent the catalog of books in the online store. Include details such as book title, author, genre, price, and availability.

Book_catalog_view.html:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Book Catalog Viewer</title>
  <link rel="stylesheet" href="book_catalog.css">
</head>
<body>
  <header class="site-header">
    <h1>Book Catalog</h1>
  </header>

  <main class="container">
    <div class="controls">
      <input id="search" placeholder="Search by title, author, or genre" />
      <select id="genreFilter"><option value="">All genres</option></select>
      <button id="refresh">Refresh</button>
    </div>

    <div id="catalog" class="grid" aria-live="polite"></div>
  </main>

  <script src="book_catalog.js"></script>
</body>
</html>
```

Book_catalog.css:

```
:root {
  --bg: #f5f3ff;
  --card: #ffffff;
  --muted: #6b7280;
  --accent: #7c3aed;
  --success: #10b981;
  --danger: #ef4444;
  --glass: rgba(255, 255, 255, 0.6);
}
* {
  box-sizing: border-box;
}
body {
```

```
  font-family:
    Segoe UI,
    Roboto,
    Arial,
    sans-serif;
  margin: 0;
  background: var(--bg);
  color: #1e1b4b;
}
.site-header {
  background: linear-gradient(90deg, var(--accent), #db2777);
  color: white;
  padding: 20px 24px;
}
.site-header h1 {
  margin: 0;
  font-size: 1.25rem;
}
.container {
  max-width: 1100px;
  margin: 24px auto;
  padding: 0 16px;
}
.controls {
  display: flex;
  gap: 10px;
  align-items: center;
  margin-bottom: 18px;
}
.controls input,
.controls select {
  padding: 10px 12px;
  font-size: 0.95rem;
  border: 1px solid #e6e9ee;
  border-radius: 8px;
  min-width: 0;
}
.controls button {
  padding: 10px 14px;
  border-radius: 8px;
  background: var(--accent);
  color: white;
  border: none;
  cursor: pointer;
}
```

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px;
}
.card {
  background: var(--card);
  padding: 14px;
  border-radius: 12px;
  box-shadow: 0 6px 20px rgba(20, 20, 40, 0.06);
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.title {
  font-weight: 700;
  font-size: 1.02rem;
}
.author {
  color: var(--muted);
  font-size: 0.92rem;
}
.meta {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 6px;
}
.genre {
  font-size: 0.85rem;
  padding: 6px 8px;
  border-radius: 999px;
  background: var(--glass);
  color: #374151;
}
.price {
  font-weight: 700;
  color: var(--accent);
}
.availability {
  font-size: 0.85rem;
  padding: 6px 8px;
  border-radius: 999px;
}
.availability.in-stock {
```

```
  background: rgba(16, 185, 129, 0.08);
  color: var(--success);
  border: 1px solid rgba(16, 185, 129, 0.12);
}
.availability.out-of-stock {
  background: rgba(239, 68, 68, 0.08);
  color: var(--danger);
  border: 1px solid rgba(239, 68, 68, 0.12);
}
.empty {
  padding: 30px;
  text-align: center;
  color: var(--muted);
}
@media (max-width: 520px) {
  .controls {
    flex-direction: column;
    align-items: stretch;
  }
  .controls input {
    width: 100%;
  }
}
```

Book_catalog.js:

```
async function loadCatalog(){
  try{
    const res = await fetch('book_catalog.xml');
    if(!res.ok) throw new Error('Failed to load XML');
    const text = await res.text();
    const parser = new DOMParser();
    const xml = parser.parseFromString(text, 'application/xml');

    const books = Array.from(xml.getElementsByTagName('book')).map(b => {
      const title = b.getElementsByTagName('title')[0]?.textContent || '';
      const author = b.getElementsByTagName('author')[0]?.textContent || '';
      const genre = b.getElementsByTagName('genre')[0]?.textContent || '';
      const priceEl = b.getElementsByTagName('price')[0];
      const price = priceEl?.textContent || '';
      const currency = priceEl?.getAttribute('currency') || '';
      const availEl = b.getElementsByTagName('availability')[0];
      const availabilityCount = availEl?.textContent || '0';
      const status = availEl?.getAttribute('status') || (Number(availabilityCount)>0 ? 'in-stock' : 'out-
of-stock');
```

```
        return { id: b.getAttribute('id'), title, author, genre, price, currency, availabilityCount: Number(availabilityCount), status };
    });

    return books;
  }catch(err){
    console.error(err);
    return null;
  }
}

function renderBooks(books){
  const container = document.getElementById('catalog');
  container.innerHTML = '';
  if(!books || books.length===0){
    container.innerHTML = '<div class="empty">No books found.</div>';
    return;
  }

  const frag = document.createDocumentFragment();
  books.forEach(b=>{
    const card = document.createElement('article');
    card.className = 'card';
    card.innerHTML = `
      <div class="title">${escapeHtml(b.title)}</div>
      <div class="author">by ${escapeHtml(b.author)}</div>
      <div class="meta">
        <div class="genre">${escapeHtml(b.genre)}</div>
        <div class="price">${escapeHtml(b.currency)} ${escapeHtml(b.price)}</div>
      </div>
      <div style="display:flex;justify-content:space-between;align-items:center">
       <div class="availability ${b.status === 'in-stock' ? 'in-stock' : 'out-of-stock'}">${b.status === 'in-stock' ? 'In stock' : 'Out of stock'}</div>
        <div style="color:var(--muted);font-size:0.85rem">Qty: ${b.availabilityCount}</div>
      </div>
    `;
    frag.appendChild(card);
  });
  container.appendChild(frag);
}

function escapeHtml(s){
  return String(s).replaceAll('&','&amp;').replaceAll('<','&lt;').replaceAll('>','&gt;');
}
```

```
function applyFilters(allBooks){
  const q = document.getElementById('search').value.trim().toLowerCase();
  const genre = document.getElementById('genreFilter').value;
  let filtered = allBooks.filter(b => {
    const hay = `${b.title} ${b.author} ${b.genre}`.toLowerCase();
    return hay.includes(q) && (genre === '' || b.genre === genre);
  });
  renderBooks(filtered);
}

(async function init(){
  const allBooks = await loadCatalog();
  if(allBooks === null){
      document.getElementById('catalog').innerHTML = '<div class="empty">Could not load catalog. Make sure <code>book_catalog.xml</code> is next to the viewer.</div>';
    return;
  }

  // populate genre filter
  const genres = Array.from(new Set(allBooks.map(b=>b.genre).filter(Boolean))).sort();
  const select = document.getElementById('genreFilter');
  genres.forEach(g=>{
    const opt = document.createElement('option');
    opt.value = g; opt.textContent = g; select.appendChild(opt);
  });

  // initial render
  renderBooks(allBooks);

  // wire controls
  document.getElementById('search').addEventListener('input', ()=>applyFilters(allBooks));
  select.addEventListener('change', ()=>applyFilters(allBooks));
  document.getElementById('refresh').addEventListener('click', async ()=>{
    const refreshed = await loadCatalog();
    if(refreshed) {
      allBooks.length = 0; allBooks.push(...refreshed);
      applyFilters(allBooks);
    }
  });
})();
```

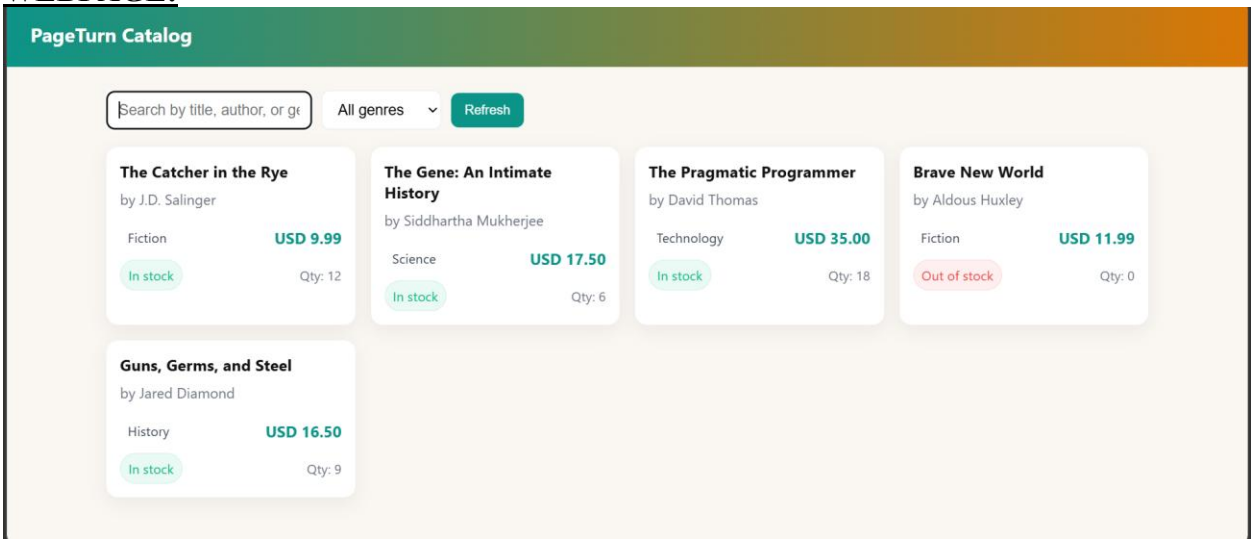
Book_catalog.xml:

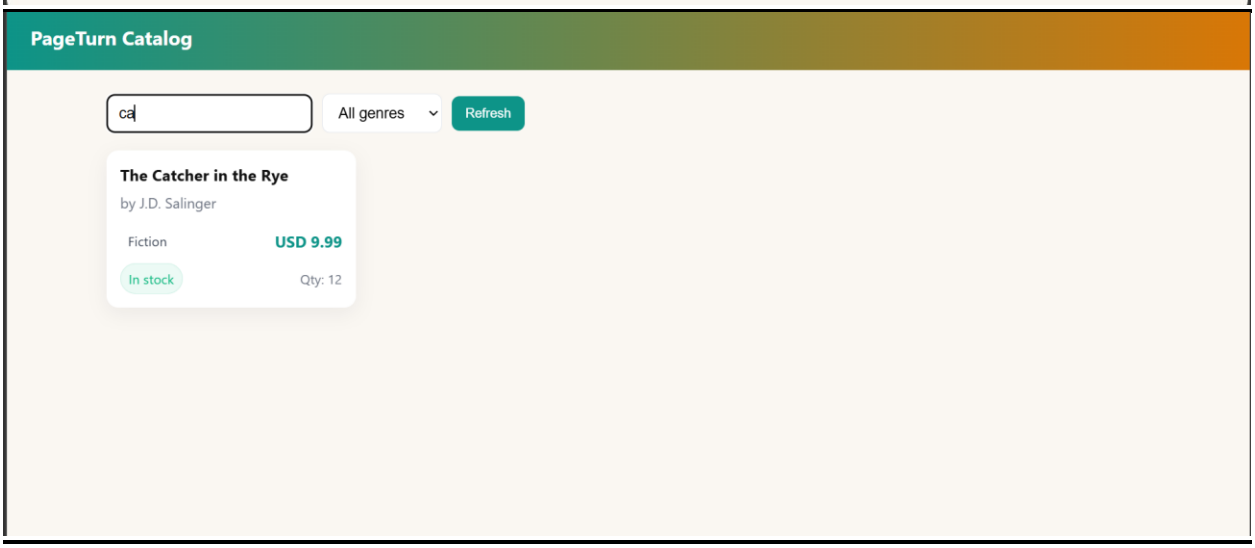
```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
   <book id="B001">
```

```
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <genre>Fiction</genre>
    <price currency="USD">10.99</price>
    <availability status="in-stock">15</availability>
  </book>

  <book id="B002">
    <title>A Brief History of Time</title>
    <author>Stephen Hawking</author>
    <genre>Science</genre>
    <price currency="USD">15.50</price>
    <availability status="in-stock">8</availability>
  </book>

  <book id="B003">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <genre>Technology</genre>
    <price currency="USD">32.00</price>
    <availability status="in-stock">23</availability>
  </book>

  <book id="B004">
    <title>To Kill a Mockingbird</title>
    <author>Harper Lee</author>
    <genre>Fiction</genre>
    <price currency="USD">12.99</price>
    <availability status="out-of-stock">0</availability>
  </book>

  <book id="B005">
    <title>Sapiens: A Brief History of Humankind</title>
    <author>Yuval Noah Harari
    <genre>History</genre>
    <price currency="USD">18.75</price>
    <availability status="in-stock">5</availability>
  </book>
</catalog>
```

DATE: EXERCISE NO: EXERCISE NAME:

WEBPAGE:





WEEK-6: User Account XML: Create an XML structure to store user account information, including details like username, password (encrypted), email, and purchase history.

1.The XML File (users.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<userAccounts>
   <user id="u1">
      <username>alice_smith</username>
<password>5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8</pass
word>
      <email>alice@example.com</email>
      <purchaseHistory>
         <purchase>
            <itemName>Wireless Keyboard</itemName>
            <price>$45.00</price>
            <date>2026-03-15</date>
         </purchase>
         <purchase>
            <itemName>USB-C Hub</itemName>
            <price>$25.00</price>
            <date>2026-04-02</date>
         </purchase>
      </purchaseHistory>
   </user>
   <user id="u2">
      <username>bob_jones</username>
<password>8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92</passwo
rd>
      <email>bob.j@example.com</email>
      <purchaseHistory>
         <purchase>
            <itemName>Ergonomic Mouse</itemName>
            <price>$60.00</price>
            <date>2026-04-05</date>
         </purchase>
      </purchaseHistory>
   </user>
</userAccounts>
```

2. Page 1: User Directory (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>User Directory</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; background: #f4f4f9; }
      .user-card { background: white; padding: 15px; margin-bottom: 10px; border-radius: 5px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
      a { text-decoration: none; color: white; background: #007bff; padding: 5px 10px; border-
radius: 3px; font-size: 14px; }
   </style>
</head>
<body>

   <h2>User Accounts Dashboard</h2>
   <div id="users-container">Loading users...</div>

   <script>
      fetch('users.xml')
         .then(response => response.text())
         .then(data => {
            const parser = new DOMParser();
            const xml = parser.parseFromString(data, "application/xml");
            const users = xml.querySelectorAll('user');
            let output = '';

            users.forEach(user => {
               const id = user.getAttribute('id');
               const username = user.querySelector('username').textContent;
               const email = user.querySelector('email').textContent;

               output += `
                  <div class="user-card">
                     <h3>${username}</h3>
                     <p>Email: ${email}</p>
                     <a href="history.html?id=${id}">View Purchase History</a>
                  </div>
               `;
            });
            document.getElementById('users-container').innerHTML = output;
         });
   </script>
```

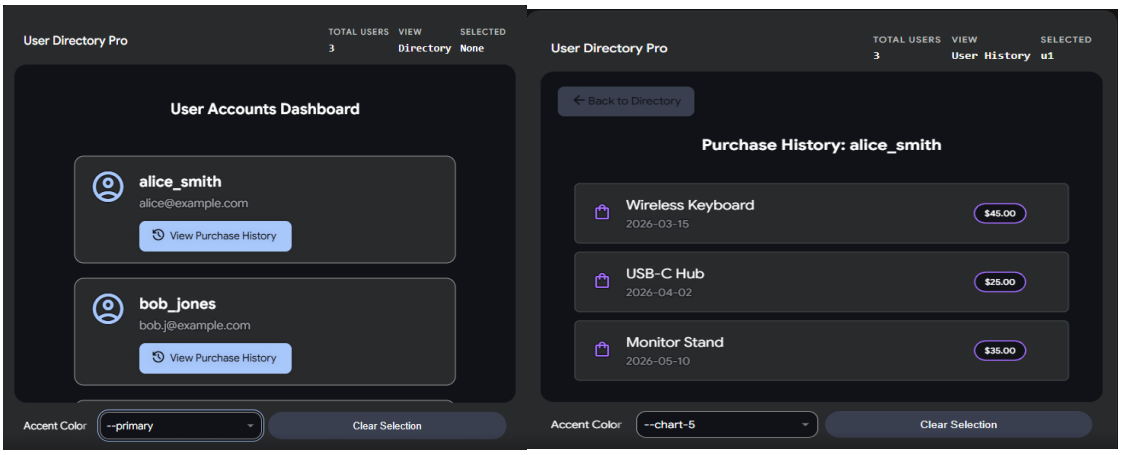
```
</body>
</html>
```

3. Page 2: Purchase History (history.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Purchase History</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; background: #f4f4f9; }
      .purchase { background: white; padding: 10px; margin-bottom: 5px; border-left: 4px solid
#28a745; }
      a.back { display: inline-block; margin-bottom: 15px; color: #007bff; text-decoration: none; }
   </style>
</head>
<body>

   <a href="index.html" class="back">← Back to Directory</a>
   <h2 id="user-title">Purchase History</h2>
   <div id="history-container">Loading...</div>

   <script>
      // Get the user ID from the URL (e.g., ?id=u1)
      const params = new URLSearchParams(window.location.search);
      const userId = params.get('id');

      fetch('users.xml')
         .then(response => response.text())
         .then(data => {
            const parser = new DOMParser();
            const xml = parser.parseFromString(data, "application/xml");

            // Find the specific user by ID
            const user = xml.querySelector(`user[id="${userId}"]`);

            if (!user) {
               document.getElementById('history-container').innerHTML = "<p>User not
found.</p>";
               return;
            }

            const username = user.querySelector('username').textContent;
```

```
            document.getElementById('user-title').textContent =`Purchase History for ${username}`;
            const purchases = user.querySelectorAll('purchaseHistory purchase');
            let output = '';

            if (purchases.length === 0) {
               output = "<p>No purchases found.</p>";
            } else {
               purchases.forEach(p => {
                  const item = p.querySelector('itemName').textContent;
                  const price = p.querySelector('price').textContent;
                  const date = p.querySelector('date').textContent;

                  output += `
                     <div class="purchase">
                        <strong>${item}</strong> - ${price} <br>
                        <small>Purchased on: ${date}</small>
                     </div>
                  `;
               });
            }
            document.getElementById('history-container').innerHTML = output;
         });
   </script>
</body>
</html>
```

OUTPUT

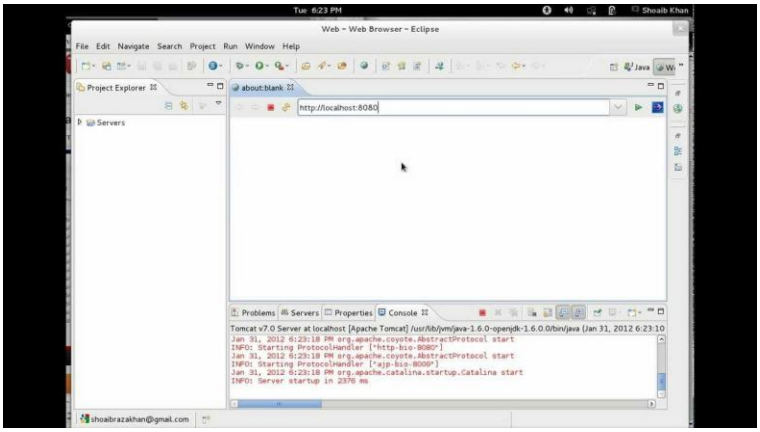


DATE: EXERCISE NO: EXERCISE NAME:

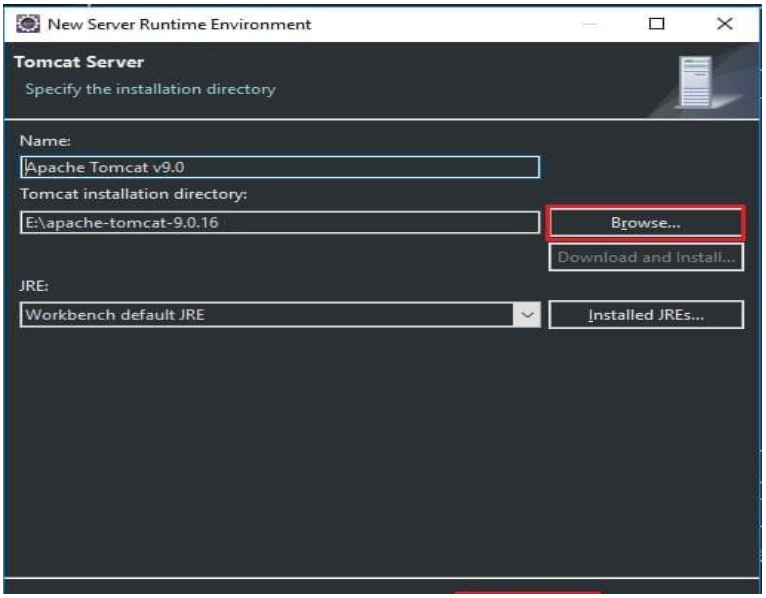
WEEK-7: Install and configure Apache Tomcat server in Eclipse

Step 1: Install Apache Tomcat (outside Eclipse)

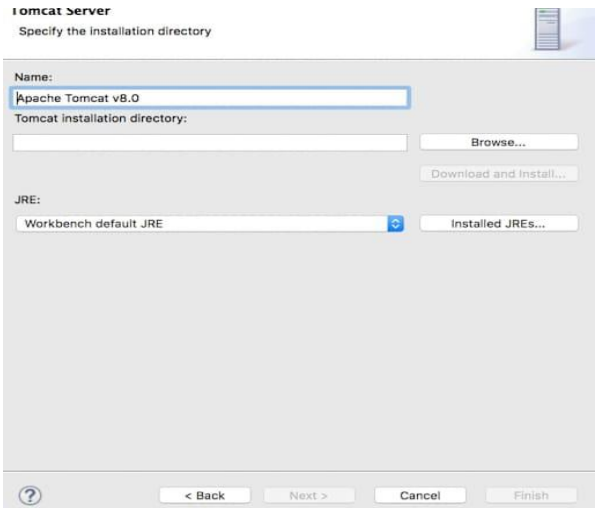
Step 2: Open Eclipse in Java EE Perspective



Step 3: Add Tomcat Server Runtime



Step 4: Create a Tomcat Server in Eclipse



Step 5: Start the Tomcat Server

Step 6: Create a Simple Web Project and Deploy on Tomcat

Step 7: Verify Everything is Working

WEEK-8: Servlet for Book Retrieval: Develop a servlet that retrieves book information from a database and sends it to the content frame.

1.Database Setup (SQL)

```
CREATE TABLE books (
   id INT PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(100),
   year INT
);

INSERT INTO books VALUES (1, 'Clean Code', 'Robert C. Martin', 2008);
INSERT INTO books VALUES (2, 'The Pragmatic Programmer', 'Andrew Hunt', 1999);
INSERT INTO books VALUES (3, 'Design Patterns', 'Erich Gamma', 1994);
```

2. Main Layout (index.html)

```
<!DOCTYPE html>
<html>
<head>
   <title>Book Library</title>
</head>
<frameset cols="20%, 80%">
   <frame src="nav.html" name="navFrame">
   <frame src="content.html" name="contentFrame">
</frameset>
</html>
```

3. Navigation Frame (nav.html)

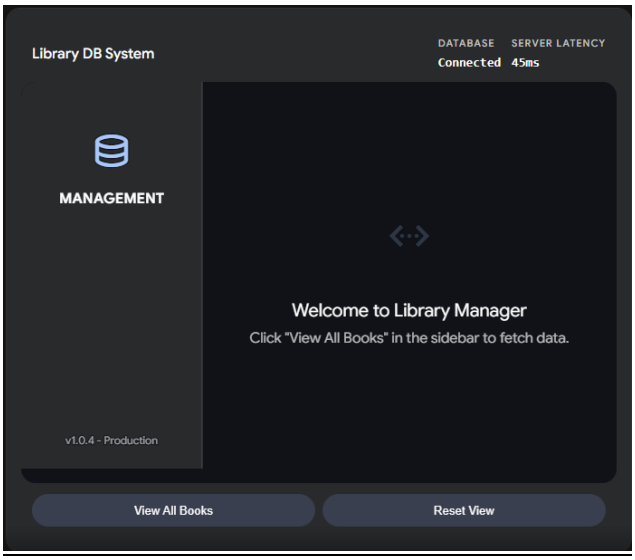
```
<!DOCTYPE html>
<html>
<body style="font-family: Arial, sans-serif; background-color: #f4f4f4;">
   <h3>Menu</h3>
   <a href="BookServlet" target="contentFrame">View All Books</a>
</body>
</html>
```

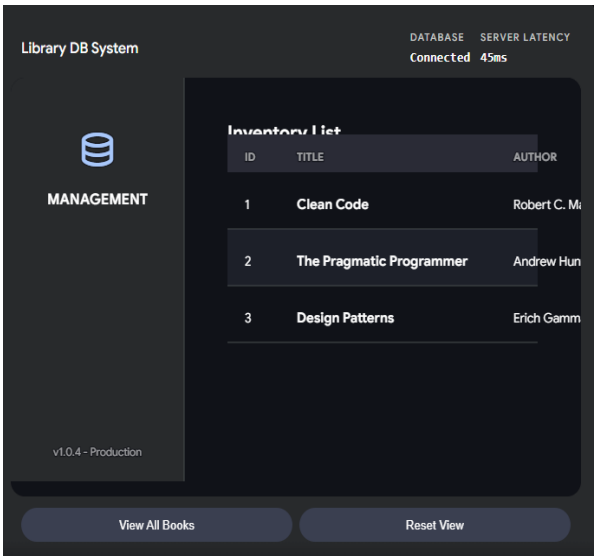
4. The Servlet (BookServlet.java)

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class BookServlet extends HttpServlet {
   public void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<html><head><style>table {width: 100%; border-collapse: collapse;} th, td
{border: 1px solid black; padding: 8px; text-align: left;} th {background-color:
#f2f2f2;}</style></head><body>");
      out.println("<h2>Book List</h2>
      try {
         // 1. Load Driver (Example for MySQL)
         Class.forName("com.mysql.cj.jdbc.Driver");

         // 2. Connect to DB (Change URL, username, password!)
         Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/yourdb",
"root", "password");

         // 3. Execute Query
         Statement stmt = con.createStatement();
         ResultSet rs = stmt.executeQuery("SELECT * FROM books");

         // 4. Print HTML Table
      out.println("<table><tr><th>ID</th><th>Title</th><th>Author</th><th>Year</th></tr>");
         while (rs.next()) {
            out.println("<tr>");
            out.println("<td>" + rs.getInt("id") + "</td>");
            out.println("<td>" + rs.getString("title") + "</td>");
            out.println("<td>" + rs.getString("author") + "</td>");
            out.println("<td>" + rs.getInt("year") + "</td>");
            out.println("</tr>");
         }
         out.println("</table>");

         con.close();
      } catch (Exception e) {
         out.println("<p>Error: " + e.getMessage() + "</p>");
      }

      out.println("</body></html>");
```

```
    }
}
```

OUTPUT


Library DB System
DATABASE
SERVER LATENCY
Connected
45ms
MANAGEMENT
Welcome to Library Manager
Click "View All Books" in the sidebar to fetch data.
v1.0.4 - Production
View All Books
Reset View


Library DB System
DATABASE
SERVER LATENCY
Connected
45ms
MANAGEMENT
ID
TITLE
AUTHOR
1
Clean Code
2
The Pragmatic Programmer
Andrew Hun
3
Design Patterns
Erich Gamm
v1.0.4 - Production
View All Books
Reset View

WEEK-9: User Authentication: Implement user authentication using servlets and JSP. Create a login page and restrict access to certain pages for authenticated users.

1.The Login Page (login.jsp)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
   <title>Login</title>
   <style>
      body { font-family: Arial, sans-serif; background: #f4f4f9; display: flex; justify-content: center; padding-top: 50px; }
      .login-box { background: white; padding: 20px; border-radius: 5px; box-shadow: 0 0 10px rgba(0,0,0,0.1); width: 300px; }
      input[type="text"], input[type="password"] { width: 90%; padding: 10px; margin: 10px 0; }
      button { width: 100%; padding: 10px; background: #007bff; color: white; border: none; cursor: pointer; }
      .error { color: red; font-size: 14px; }
   </style>
</head>
<body>
   <div class="login-box">
      <h2>Sign In</h2>
      <% if(request.getAttribute("error") != null) { %>
         <p class="error"><%= request.getAttribute("error") %></p>
      <% } %>
      <form action="LoginServlet" method="post">
         <input type="text" name="username" placeholder="Username" required><br>
         <input type="password" name="password" placeholder="Password" required><br>
         <button type="submit">Login</button>
      </form>
   </div>
</body>
</html>
```

2. The Authentication Servlet (LoginServlet.java)

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        // Hardcoded check (Replace with DB validation in production)
        if ("admin".equals(user) && "password123".equals(pass)) {
            // Create a session and store the username
            HttpSession session = request.getSession();
            session.setAttribute("username", user);
            // Redirect to the restricted page
            response.sendRedirect("dashboard.jsp");
        } else {
            // Send back to login page with an error
            request.setAttribute("error", "Invalid username or password");
            request.getRequestDispatcher("login.jsp").forward(request, response);
        }
    }
}
```

3. The Restricted Page (dashboard.jsp)

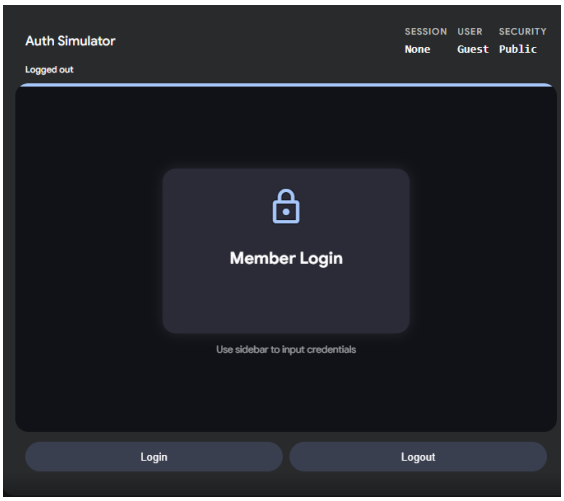
```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%
    // Security Check: Restrict access to authenticated users only
    if (session.getAttribute("username") == null) {
        response.sendRedirect("login.jsp");
        return; // Stop rendering the page
    }
%>
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard - Restricted Area</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 20px; text-align: center; }
        .card { background: #e0f7fa; padding: 20px; display: inline-block; border-radius: 8px; }
        a.logout { color: white; background: #dc3545; padding: 8px 15px; text-decoration: none; border-radius: 4px; }
    </style>
</head>
<body>
    <div class="card">
        <h2>Welcome, <%= session.getAttribute("username") %>!</h2>
        <p>You have successfully accessed the restricted dashboard.</p>
```

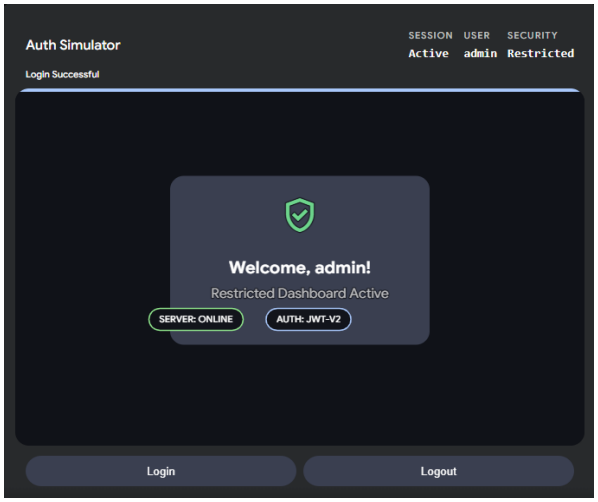
```
        <br>
        <a href="LogoutServlet" class="logout">Logout</a>
    </div>
</body>
</html>
```

4. The Logout Servlet (LogoutServlet.java)

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false); // Get existing session
        if (session != null) {
            session.invalidate(); // Destroy the session
        }
        response.sendRedirect("login.jsp");
    }
}
```

OUTPUT





WEEK-10:
Hibernate: Hibernate Entity Mapping: Create Hibernate entity classes to represent books and users. Map these entities to corresponding database tables

1.The Book Entity (Book.java)

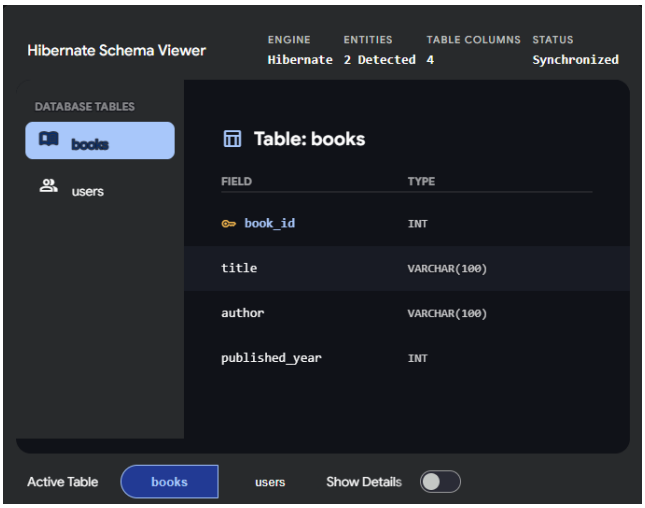
```
import javax.persistence.*;
@Entity
@Table(name = "books")
public class Book {
   @Id // Marks this as the Primary Key
   @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increments the ID
   @Column(name = "book_id")
   private int id;
   @Column(name = "title", nullable = false, length = 100)
   private String title;
   @Column(name = "author", length = 100)
   private String author;
   @Column(name = "published_year")
   private int year;
   // Default constructor is required by Hibernate
   public Book() {}
   // Getters and Setters
   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   public String getTitle() { return title; }
   public void setTitle(String title) { this.title = title; }
   public String getAuthor() { return author; }
   public void setAuthor(String author) { this.author = author; }
   public int getYear() { return year; }
   public void setYear(int year) { this.year = year; }
}
```

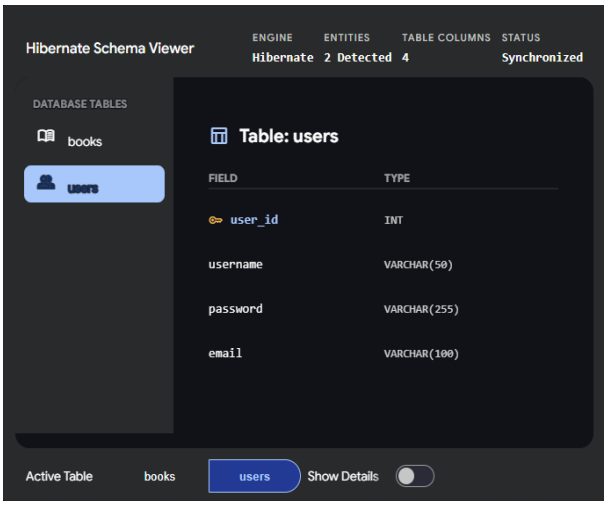
2. The User Entity (User.java)

```
import javax.persistence.*;
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "user_id")
   private int id;
   @Column(name = "username", unique = true, nullable = false, length = 50)
   private String username;
```

```
    @Column(name = "password", nullable = false) // Passwords should ideally be hashed
    private String password;
    @Column(name = "email", length = 100)
    private String email;
    // Default constructor is required by Hibernate
    public User() {}
    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

OUTPUT





WEEK-11:
CRUD Operations: Implement CRUD operations using Hibernate for managing book information in the database.

1.The Database Helper (HibernateUtil.java)

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            // Loads configuration and builds the session factory
            sessionFactory = new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

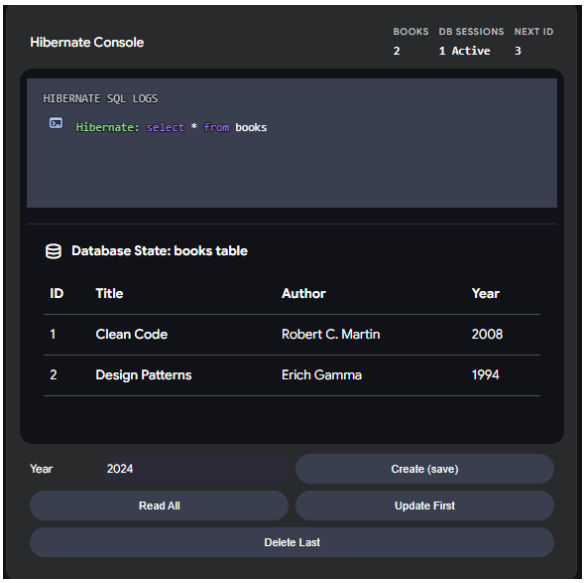
2. The CRUD Implementation (BookDAO.java)

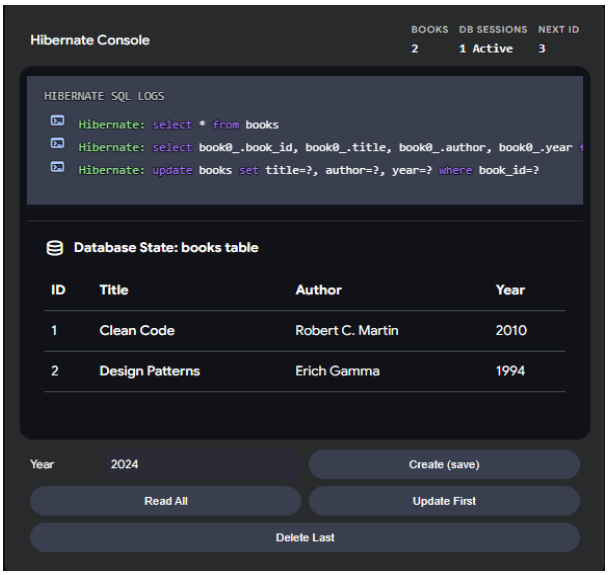
```
import org.hibernate.Session;
import org.hibernate.Transaction;
import java.util.List;
public class BookDAO {
    // -------------------------------------------------------
    // CREATE: Add a new book to the database
    // -------------------------------------------------------
    public void saveBook(Book book) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.save(book); // Hibernate automatically writes the INSERT SQL
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback(); // Undo changes if something goes wrong
            }
            e.printStackTrace();
```

```
        }
    }

    // ---------------------------------------------------------
    // READ: Get a specific book by its ID
    // ---------------------------------------------------------
    public Book getBook(int id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Book.class, id); // Hibernate writes the SELECT SQL
        }
    }

    // ---------------------------------------------------------
    // READ: Get a list of all books in the database
    // ---------------------------------------------------------
    @SuppressWarnings("unchecked")
    public List<Book> getAllBooks() {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            // HQL (Hibernate Query Language) uses the Java class name, not the DB table name
            return session.createQuery("from Book").list();
        }
    }

    // ---------------------------------------------------------
    // UPDATE: Modify an existing book's details
    // ---------------------------------------------------------
    public void updateBook(Book book) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.update(book); // Hibernate writes the UPDATE SQL
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback();
            }
            e.printStackTrace();
        }
    }

    // ---------------------------------------------------------
    // DELETE: Remove a book from the database
    // ---------------------------------------------------------
    public void deleteBook(int id) {
        Transaction transaction = null;
```

```
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();

            // First, fetch the book we want to delete
            Book book = session.get(Book.class, id);
            if (book != null) {
                session.delete(book); // Hibernate writes the DELETE SQL
            }

            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback();
            }
            e.printStackTrace();
        }
    }
}
```

OUTPUT





WEEK-12: Full-Stack Integration: Integrate JavaScript with Servlets: Enhance the shopping cart functionality by integrating JavaScript with servlets. Allow users to update the cart without page refresh.

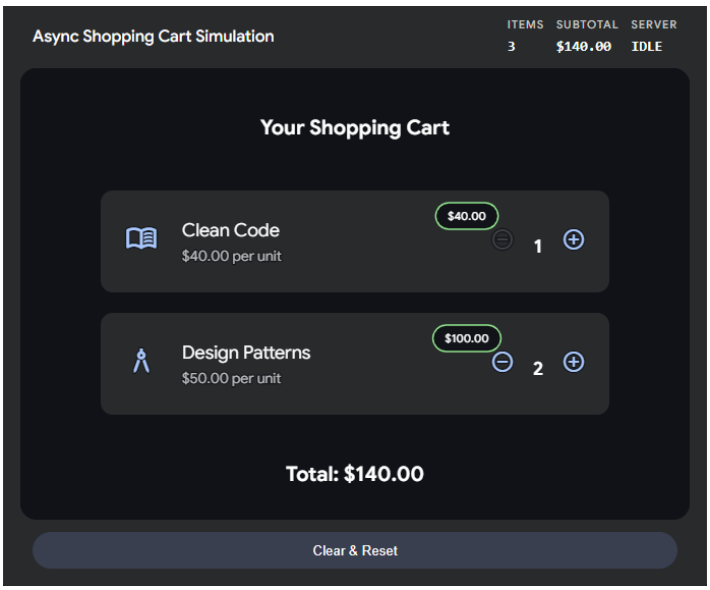
1.The Frontend (cart.jsp)

```
<!DOCTYPE html>
<html>
<head>
   <title>AJAX Shopping Cart</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; max-width: 500px; }
      .cart-item { padding: 10px; border-bottom: 1px solid #ddd; display: flex; justify-content:
space-between; align-items: center; }
      .cart-item input { width: 50px; padding: 5px; }
      .status-message { color: #28a745; font-size: 14px; display: none; margin-bottom: 10px; }
      .total-row { margin-top: 20px; font-size: 1.2em; font-weight: bold; text-align: right; }
   </style>
</head>
<body>

   <h2>Your Shopping Cart</h2>
   <div id="updateStatus" class="status-message">Cart updated successfully!</div>

   <div class="cart-item">
      <span>Clean Code ($40.00)</span>
      <input type="number" value="1" min="1" onchange="updateCart(1, 40.00, this.value)">
   </div>
   <div class="cart-item">
      <span>Design Patterns ($50.00)</span>
      <input type="number" value="1" min="1" onchange="updateCart(2, 50.00, this.value)">
   </div>

   <div class="total-row">
      Total: $<span id="cartTotal">90.00</span>
   </div>

   <script>
      // JavaScript for asynchronous updating (AJAX)
      function updateCart(bookId, price, quantity) {
         // Send a background request to the Servlet
         fetch('CartServlet', {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
```

```
            body: `bookId=${bookId}&price=${price}&quantity=${quantity}`
        })
        .then(response => response.json()) // The Servlet will return JSON data
        .then(data => {
            // Update the total on the screen dynamically
            document.getElementById('cartTotal').innerText = data.newTotal;

            // Flash a success message
            let status = document.getElementById('updateStatus');
            status.style.display = 'block';
            setTimeout(() => { status.style.display = 'none'; }, 2000); // Hide after 2 seconds
        })
        .catch(error => console.error('Error updating cart:', error));
    }
  </script>
</body>
</html>
```

2. The Backend (CartServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/CartServlet")
public class CartServlet extends HttpServlet {
   protected void doPost(HttpServletRequest request, HttpServletResponse response)
          throws ServletException, IOException {
      // 1. Read the parameters sent by Javascript
      int bookId = Integer.parseInt(request.getParameter("bookId"));
      double price = Double.parseDouble(request.getParameter("price"));
      int quantity = Integer.parseInt(request.getParameter("quantity"));
      // 2. Business Logic: Update session cart and recalculate total
      // Note: For this isolated example, we are just calculating the total of the changed item.
      // In a real app, you would retrieve the user's cart object from the session and sum all items.
      HttpSession session = request.getSession();
      // Simulating a cart total calculation (Assuming they only have this one item type for the demo
math)
      // You would replace this with actual cart iteration logic.
      double simulatedNewTotal = price * quantity;
      // 3. Set the response content type to JSON
      response.setContentType("application/json");
      response.setCharacterEncoding("UTF-8");

      // 4. Send the JSON response back to the Javascript fetch() call
```

```
        PrintWriter out = response.getWriter();
        out.print("{\"newTotal\": \"" + String.format("%.2f", simulatedNewTotal) + "\"}");
        out.flush();
    }
}
```

OUTPUT